BrainAI

ProSuite Industrial

HARDWARE ACCELERATED HASHING

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**FPGA & ASIC IP Core Guide**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie / Revision History

Version	Datum	Autor	Status	Beschreibung
1.0.0	01.01.2025	S. Köhne	Stable	Initial Hardware & CLI Documentation
2.0.0	27.02.2026	S. Köhne	Stable	Public Release inkl. Internal Deep Dive & Register Maps

1 Produktübersicht & Philosophie

Das **ProHash-256 Hardware Security Module (HSM)** lagert die Berechnung kryptographisch sicherer Prüfsummen physisch auf einen FPGA-Chip (Field Programmable Gate Array) aus.

Souveränität & Zero-Trust: Die Berechnung findet vollständig getrennt vom Betriebssystem statt. Malware auf dem Host-PC (Windows/Linux) kann den Prozess durch Denial-of-Service stören, jedoch niemals den internen kryptographischen Zustand des Chips auslesen oder den Algorithmus manipulieren.

2 Hardware-Architektur (The Deep Dive)

Die FPGA-Implementierung ist wie eine Fabrik strukturiert, die in drei voneinander getrennte Module unterteilt ist:

2.1 Die Komponenten

1. **`prohash_top.v` (Warenannahme & I/O):** Das Top-Level-Modul. Es verwaltet die externen Pins, empfängt die UART/USB-Pakete vom PC und sendet die fertigen Hash-Werte zurück.
2. **`prohash_soc_wrapper.v` (Speicher & Register):** Die Brücke. Es decodiert die eingehenden Befehle und sortiert die 64-Byte-Datenblöcke in die entsprechenden Input-Register.
3. **`prohash_core.v` (Der "Grinder"):** Die mathematische ALU. Hier befindet sich das Sponge-Konstrukt (`S_GRIND`), welches die physikalische Bit-Mischung vornimmt.

2.2 Anpassungen (Safe Zone)

- **Übertragungsrate:** Standardmäßig auf `115200` Baud konfiguriert. Bei hochqualitativen Kabeln kann dies in der `prohash_top.v` und `uart_rx.v` auf `921600` angehoben werden. Hardware und C-Treiber müssen stets denselben Wert aufweisen.
- **LED-Status:** Die Zuweisung der Pins (`assign leds[...]`) kann gefahrlos an das verwendete FPGA-Board (z.B. für einen `tx_busy` Indikator) angepasst

werden.

2.3 Kritische Betriebsgeheimnisse (Danger Zone)

Veränderungen an folgenden Stellen führen zur Zerstörung der mathematischen Integrität:

- **Konstanten:** Die Initial-Vektoren wie `C_MUL1 = 64'hff51afd7ed558ccd` dürfen niemals modifiziert werden. Eine Änderung resultiert in Inkompatibilität zur Software-Suite.
- **Die `S_GRIND`-Schleife:** Modifikationen an den logischen Gattern (z.B. Ersetzen von XOR ^ durch ADD +) korrumpieren den Sponge-Mechanismus irreversibel.

3 Physical Layer & Memory Map

Die Kommunikation zwischen Host-PC und FPGA nutzt ein binäres Memory-Mapped I/O Protokoll über eine 8N1 UART-Verbindung (Little Endian).

3.1 Das UART-Protokoll

- **Schreiben (CMD `0x01`):** `[0xA5] [0x01] [ADDR] [LSB] [..] [..] [MSB]`. Schreibt 4 Bytes in ein 32-Bit Register.
- **Lesen (CMD `0x02`):** `[0xA5] [0x02] [ADDR]`. Der FPGA antwortet unmittelbar mit 4 Bytes (LSB first).

3.2 Die Registerkarte

Der FPGA stellt folgende Speicheradressen für den Zugriff bereit:

Adresse	Name	R/W	Beschreibung
`0x00`	**PH_REG_CTRL**	W	Bit 0: INIT (Reset Core). Bit 1: ABSORB (Start Grind).
`0x01`	**PH_REG_STATUS**	R	Bit 0: READY (Bereit). Bit 1: BUSY (Rechnet).
`0x10` - `0x1F`	**INPUT BUFFER**	W	16 Wörter (64 Bytes) für den aktuellen Datenblock.
`0x20` - `0x27`	**OUTPUT HASH**	R	8 Wörter (32 Bytes) für das fertige 256-Bit Ergebnis.

4 CLI User Guide (prohash_cli)

Das Command-Line-Interface orchestriert die Low-Level-Register-Befehle zu einem bedienbaren Endanwender-Tool für Windows und Linux.

4.1 System-Voraussetzungen & Identifikation

- **Windows:** Der FPGA meldet sich als "USB Serial Port" oder "Silicon Labs CP210x" im Geräte-Manager (z.B. `COM3`).
- **Linux:** Im Terminal über `dmesg | grep tty` auslesbar (z.B. `/dev/ttyUSB0`). Für den Zugriff muss der Linux-Benutzer in der `dialout` Gruppe sein (`sudo usermod -a -G dialout $USER`).

4.2 Standard-Benutzung

Die Syntax lautet: `prohash_cli [ANSCHLUSS] [DATEIPFAD]`

Beispiel für Linux:

```
./prohash_cli /dev/ttyUSB0 /tmp/system_image.iso
# [*] Opening Hardware Link on /dev/ttyUSB0... OK.
# [*] Initializing FPGA Core... READY.
# [*] Grinding file...
# ---------------------------------------------------
# ProHash-256: 8f4b2e1a9c...
```

5 Fehlerbehebung & Hardware-Troubleshooting

Sollte die Kommunikation fehlschlagen, liefert die CLI direkte Anhaltspunkte:

- **"Could not open serial port":** Der angegebene COM-Port ist falsch, oder er wird exklusiv von einer anderen Software (z.B. Putty, Arduino IDE) blockiert.
- **Hänger bei "Initializing FPGA Core...":** Der PC sendet das SYNC-Byte (`0xA5`), erhält aber keine Antwort vom FPGA.
- *Lösung A:* FPGA über den Hardware-Taster resetten.
- *Lösung B:* Bei externen UART-Adaptern prüfen, ob **TX und RX überkreuzt** verbunden sind.
- **Inkonsistente Hash-Werte:** Wenn dieselbe Datei wechselnde Hashes erzeugt, ist die USB-Leitung defekt. ProHash ist physikalisch deterministisch. Verwenden Sie ein kürzeres, geschirmtes Kabel.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**